**UNIVERSIDADE FEDERAL DE SANTA CATARINA**
**DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA**
**CURSO DE CIÊNCIAS DA COMPUTAÇÃO**

**ALEX DAVIS NEUWIEM DA SILVA**
**LUAN DINIZ MORAES**
**PEDRO NACK MARTINS**

# Criptografia Homomórfica - O Sistema de ElGamal

**Florianópolis, 2024.**

**UNIVERSIDADE FEDERAL DE SANTA CATARINA**
**DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA**
**CURSO DE CIÊNCIAS DA COMPUTAÇÃO**

**ALEX DAVIS NEUWIEM DA SILVA**
**LUAN DINIZ MORAES**
**PEDRO NACK MARTINS**

# Criptografia Homomórfica - O Sistema de ElGamal

Relatório submetido à Unidade Curricular de Segurança em Computação, da 7ª fase do Curso de Graduação em Ciências da Computação da Universidade Federal de Santa Catarina, Câmpus Trindade.

Professores:
Jean Everson Martina
Thaís Bardini Idalino

**Florianópolis, 2024.**

# 1. INTRODUÇÃO

O algoritmo escolhido para este relatório foi o sistema criptográfico parcialmente homomórfico de ElGamal.

Ele é dito *parcialmente homomórfico* pois o ElGamal suporta apenas um tipo de operação, embora ela possa ser aplicada quantas vezes for necessário [10].

Embora parcialmente homomórfico, uma pequena alteração no algoritmo de encriptação do Elgamal torna ele homomórfico em relação à adição ou à multiplicação. Esse foi um dos motivadores da nossa escolha, sua versatilidade.

Pensamos em uma arquitetura e em uma aplicação, a qual será implementada futuramente.

A seguir, será dado mais contexto e também embasamento básico sobre os temas discorridos neste trabalho.

## 1.1. Conceitos Básicos

O objetivo deste tópico é apresentar os conceitos básicos que são fundamentais para o entendimento do sistema criptográfico homomórfico de ElGamal. Todas as explicações apresentadas a seguir se baseiam na referência [9].

### 1.1.1 Álgebra Abstrata

A Álgebra Abstrata, ou Álgebra Moderna, é um ramo da matemática capaz de revelar princípios gerais que se aplicam para todas as álgebras. Por exemplo, a álgebra booleana é um tipo específico de álgebra, assim como a álgebra elementar é outro, porém as regras definidas pela Álgebra Abstrata valem para ambas. (A partir desse momento, chamaremos a Álgebra Abstrata no texto simplesmente como Álgebra).

A Álgebra é definida como sendo o estudo de *estruturas algébricas*. Uma estrutura algébrica é um conjunto arbitrário com pelo menos uma *operação* definida nele.

Uma operação $*$ em $A$ é definida como uma regra que é atribuída para cada par ordenado $(a,\ b)$ de elementos em $A$ exatamente um elemento $a\ *\ b$ em $A$. Além disso, as seguintes regras são respeitadas:

- $a\ *\ b$ está definido para cada par ordenado $(a,\ b)$ de elementos de $A$.
- $a\ *\ b$ precisa ser definido de forma única.
- $A$ é fechado sobre a operação $*$.

### 1.1.2 Grupos

Um grupo é uma das estruturas algébricas mais simples que existem. Basicamente, ele é um conjunto $G$ com uma operação $*$ que satisfaz os seguintes axiomas:

- $*$ é associativo.
- Existe um elemento neutro $e$ em $G$, de tal forma que $a * e = a$ e $e * a = a$.
- Para cada elemento a em $G$, existe uma inversa $a^{-1} * a = e$ e $a * a^{-1} = e$.

A notação $< G, * >$ representa um grupo que consiste no conjunto $G$ e a operação $*$.

Um grupo em que sua operação é também comutativa é chamado de *grupo abeliano*.

O conjunto $Z_n = \{0, 1, ..., n - 1\}$ com a operação de adição modular é um exemplo de grupo.

### 1.1.3 Grupos Cíclicos

Considere o grupo $G = < A, * >$, sendo que $a$ é um elemento de $A$.

Podemos definir uma potência de $a$:

$$a^n = a * a * ... * a \quad (n \text{ vezes})$$

Pode ser que todo elemento de $G$ seja uma potência de $a$:

$$G = \{a^n : n \in A\}$$

Nesse caso, $G$ é chamado de um *grupo cíclico,* enquanto $a$ é seu gerador:

$$G = < a >$$

A ordem de um elemento de um grupo é definida como:

$$a^n = e$$

Sendo $n$ um número *positivo* e o menor possível (note que 0 não é um número positivo).

No caso acima, o elemento $a$ tem ordem $n$.

A ordem de um grupo ($|G|$) é o número de elementos que esse grupo possui, grupos podem ter ordem finita ou infinita.

Como $a$ tem ordem $n$ e é gerador de $G$, $|G|$ também é igual a $n$.

### 1.2 Homomorfismo na Criptografia

Homomorfismo, na criptografia, é um termo usado para definir uma característica muito poderosa presente em alguns algoritmos criptográficos: a capacidade de realizar operações sobre mensagens criptografadas. Tendo esse poder em mãos, não se faz necessário descriptografar uma informação para realizar computações com ela, e isso pode ser crucial para a manutenção da anonimidade ou do sigilo da mensagem.

Matematicamente, homomorfismo é uma estrutura que preserva mapeamento entre duas estruturas algébricas. Por exemplo, considere uma função $f$:

$$f\colon \mathbb{Z} \to \mathbb{Z},\ f(x) = x + x$$

Usando como exemplo a operação de soma, temos que:

$$f(a) + f(b) = f(a + b)$$

De forma genérica, temos que um algoritmo de criptografia E precisa se enquadrar na definição:

$$E(a) \oplus E(b) = E(a \otimes b)$$

Sendo $\oplus$ e $\otimes$ os operadores lógicos.

De acordo com a IEEE [7], uma demonstração sobre os benefícios do homomorfismo pode ser vista no contexto do armazenamento de informações em nuvem. Ao utilizar um algoritmo de criptografía homomórfica, podemos guardar todos os dados criptografados na nuvem e, para realizar uma ação como uma busca por índice, não é necessário descriptografá-los, protegendo-os de vulnerabilidades na nuvem.

Ademais, uma possível aplicação de algoritmos homomórficos é em um sistema de votação secreta. Pode ser feita, por exemplo, a contagem dos votos dos participantes de forma criptografada e, no fim, somente o resultado é descriptografado, protegendo assim a privacidade dos envolvidos. A tarefa desenvolvida na entrega prática deste trabalho demonstrará o poder deste tipo de ferramenta.

## 2. SISTEMA DE ELGAMAL COM HOMOMORFISMO PELA MULTIPLICAÇÃO

A demonstração apresentada nos tópicos 2.1, 2.2 e 2.3 foi retirada de [1] e [2]. A seguir, serão detalhados os processos de geração de par de chaves, cifragem de uma mensagem e recuperação de uma mensagem cifrada.

### 2.1. Gerando o par de chaves

A primeira entidade, representada pela Alice, gera um par de chaves da seguinte maneira:

- Gere uma descrição eficiente de um grupo cíclico $G$ de ordem $q$ e gerador $g$. Considere $e$ como o elemento identidade de $G$.
- Escolha aleatoriamente um inteiro $x$ do conjunto $\{1, \ldots, q-1\}$.
- Calcule $h := g^x$.

A chave pública consiste nos valores $(G, q, g, h)$. Alice distribui essa chave pública e retém $x$ como sua chave privada, que deve ser mantida em segredo.

Não é necessário criar um grupo e um gerador para cada nova chave. Na verdade, pode-se esperar que uma implementação específica do ElGamal seja codificada para usar um grupo específico, ou um grupo de um conjunto específico. A escolha do grupo depende principalmente do tamanho das chaves que você deseja usar.

### 2.2. Criptografando a mensagem

A segunda entidade, representada pelo Bob, cifra a mensagem $M$ destinada à Alice usando a chave pública dela. Essa cifragem ocorre como descrito a seguir:

- Mapeie a mensagem $M$ para um elemento $m$ de $G$ usando uma função de mapeamento reversível.
- Escolha aleatoriamente um inteiro $y$ do conjunto $\{1, \ldots, q-1\}$.
- Calcule $s := h^y$. Isso é chamado de segredo compartilhado.
- Calcule $c_1 := g^y$.
- Calcule $c_2 := m * s$.

Bob envia o texto cifrado $(c_1, c_2)$ para Alice. Note que se alguém souber qual o texto cifrado $(c_1, c_2)$ e a mensagem não cifrada $m$, será fácil descobrir qual o segredo compartilhado $s$, já que $c_2 * m^{-1} = s$. Por causa disso, um novo $y$ e, portanto, um novo $s$ são gerados em toda nova mensagem para melhorar a segurança. Por essa razão, $y$ é chamado de chave efêmera.

### 2.3. Recuperando a mensagem criptografada

Alice decifra a mensagem $(c_1, c_2)$ com sua chave privada $x$ da seguinte maneira:

- Calcule $s := c_1^{x}$. Como $c_1 = g^y$ e $c_1^{x} = g^{xy} = h^y$, portanto é o mesmo segredo compartilhado que foi usado por Bob.
- Calcule $s^{-1}$, a inversa de $s$ no grupo $G$. Esse cálculo pode ser realizado de diversas maneiras diferentes. Se $G$ é um subgrupo de um grupo multiplicativo módulo $n$, onde $n$ é primo, o inverso multiplicativo modular pode ser calculado por meio do algoritmo extendido de Euclides. Outra alternativa é calcular $s^{-1}$ como $c_1^{q-x}$. Esse é o inverso de $s$ por causa do teorema de Lagrange já que $s * c_1^{q-x} = g^{xy} * g^{(q-x)y} = (g^q)^y = e^y = e$.
- Calcule $m := c_2 * s^{-1}$. Esse cálculo resulta na mensagem original $m$ porque $c_2 := m * s$, portanto $c_2 * s^{-1} = (m * s) * s^{-1} = m * e = m$.
- Mapeie $m$ de volta para a mensagem original $M$.

### 2.4. Exemplo prático

O exemplo demonstrado a seguir foi retirado de Stinson and Paterson [8] e demonstra a funcionalidade do sistema de ElGamal com homomorfismo pela

multiplicação. Para este exemplo, primeiramente vamos escolher um número primo $p$ e selecionar um outro número primo $q$ tal que $q \mid (p - 1)$. O grupo $G$ será o subgrupo cíclico de ordem $q$ no grupo multiplicativo dos inteiros módulo $p$.

### 2.4.1 Escolhendo os parâmetros

- $p = 2579$
- $q = 2578$
- $g = 2$ (gerador de ordem 2578 no grupo cíclico $G$)
- $x = 765$
- $h = g^x = 2^{765} \, mod \, 2579 = 949$ (devemos usar $mod \, p$ pois estamos operando no grupo cíclico dos inteiros módulo 2579)

### 2.4.2 Criptografando a mensagem $m$

Primeiramente, vamos supor que Alice deseja enviar a mensagem $m = 1299$ (elemento de $G$) para Bob.

- $y = 853$(número aleatório e elemento de $G$)
- $c_1 = g^y \, mod \, p$

  $c_1 = 2^{853} \, mod \, 2579 = 435$
- $c_2 = m * h^y \, mod \, p$

  $c_2 = 1299 * 949^{853} \, mod \, 2579 = 2396$

Mensagem criptografada: $M = (435, 2396)$.

### 2.4.3 Recuperando a mensagem cifrada

Quando Bob receber a mensagem $M = (435, 2396)$, ele calcula:

- $m = c_2 * (c_1^x)^{-1} mod\, p$

  $m = 2396 * (435^{765})^{-1} mod\, 2579 = 1299$

Com isso, Bob obtém 1299, que é a mensagem que Alice cifrou.

## 3. HOMOMORFISMO POR ADIÇÃO X HOMOMORFISMO POR MULTIPLICAÇÃO

Os conceitos e provas apresentados nesta seção foram extraídos de [6].Os fundamentos teóricos do homomorfismo se baseiam na realização de operações entre dois valores estruturados algebricamente da mesma forma, tendo um resultado que segue a mesma estrutura.

Exemplos de operação são:

- **Adição**

$$f(x * y) = f(x) + f(y)$$

- **Multiplicação**

$$f(x * y) = f(x) * f(y)$$

Neste trabalho, faremos o uso da criptografia de ElGamal. Como previamente exposto, temos que:

- $E = Função\ ElGamal$
- $m = Mensagem$
- $g = Gerador$
- $r = Inteiro\ aleatório$
- $x = Chave\ privada$
- $h = Chave\ pública$

Então, para cifragem usando ElGamal:

- $E(m) = (c_1, c_2) \rightarrow E(m) = (g^r, m * h^r)$

Para provar que o algoritmo possui a propriedade da multiplicação homomórfica, seguimos os seguintes passos:

- $E(m_1) * E(m_2) \rightarrow E(g^{r_1}, m_1 * h^{r_1}) * E(g^{r_2}, m_2 * h^{r_2}) \rightarrow$
  $E(g^{r_1+r_2}, (m_1 * m_2) * h^{r_1+r_2}) \rightarrow E(m_1 * m_2)$

Logo, observa-se que multiplicar as mensagens antes da criptografia tem o mesmo efeito de multiplicar os dados já criptografados, indicando sucesso no homomorfismo da operação.

Para demonstrar o homomorfismo aditivo, é necessário fazer pequenas alterações no algoritmo. Desta vez, a mensagem m vai ser primeiro transformada

em $g^m$. Depois, o algoritmo segue como definido nas seções anteriores. Com isso, temos que:

- $E(m) = (g^r, g^m * h^r)$

Seguindo para a prova:

- 1. $E(m_1) * E(m_2) = E(m_1 + m_2)$
- 2. $(g^{r_1}, g^{m_1} * h^{r_1}) * (g^{r_2}, g^{m_2} * h^{r_2})$
- 3. $(g^{r_1+r_2}, g^{m_1} * g^{m_2} * h^{r_1+r_2})$
- 4. $(g^{r_1+r_2}, g^{m_1+m_2} * h^{r_1+r_2})$
- 5. $E(m_1 + m_2)$

- $E(m_1) * E(m_2) = E(m_1 + m_2) \rightarrow$
  $(g^{r_1}, g^{m_1} * h^{r_1}) * (g^{r_2}, g^{m_2} * h^{r_2}) \rightarrow$
  $(g^{r_1+r_2}, g^{m_1} * g^{m_2} * h^{r_1+r_2}) \rightarrow$
  $(g^{r_1+r_2}, g^{m_1+m_2} * h^{r_1+r_2}) \rightarrow E(m_1 + m_2)$

Assim, é demonstrada a igualdade de realizar a operação antes e depois da criptografia da mensagem, apresentando característica de homomorfismo em relação à adição.

Note que, com as alterações feitas no algoritmo de ElGamal base para satisfazer a propriedade de homomorfismo aditivo, é necessário resolver o problema $g^m$ para recuperar a mensagem no final do processo. Temos então o problema do logaritmo discreto, o que faz com que essa versão do algoritmo só seja efetiva quando trabalhamos com valores pequenos, ou temos uma mensagem computacionalmente difícil de ser descriptografada, indo contra qualquer princípio do que estamos tentando realizar.

## 4. IMPLEMENTAÇÃO PRÁTICA COM PYTHON

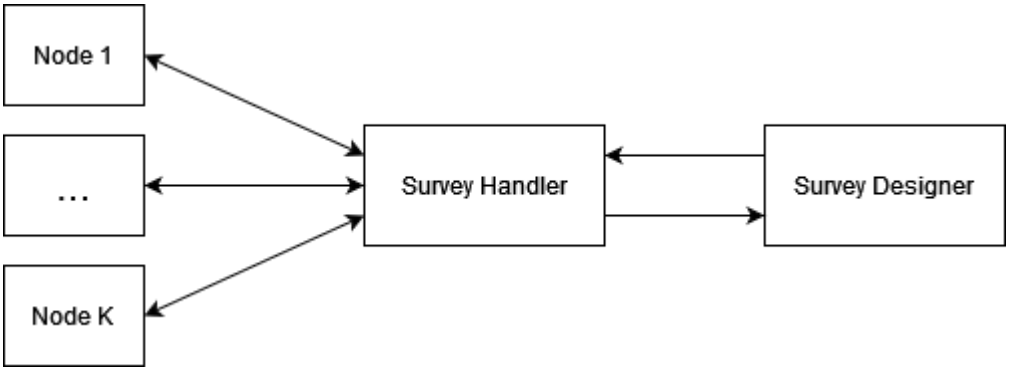
Será construída uma aplicação em Python que ilustra o funcionamento do ElGamal e sua possível utilidade. Utilizar-se-á LightPHE [4], a qual é uma biblioteca que, entre outros esquemas, já implementa o ElGamal.

A ideia da aplicação consiste em criar um sistema distribuído que representa o cenário de alguém criando um questionário anônimo para outros responderem.

Essa ideia é relevante considerando contextos que envolvam relações de hierarquia. Por exemplo, uma empresa ou um professor criando um questionário para seus funcionários ou alunos responderem.

Normalmente, se um questionário é declarado anônimo é porque a não anonimidade das respostas pode afetar negativamente de alguma forma os respondentes.

Com isso em mente, foi pensada uma arquitetura cujo objetivo consiste em garantir essa anonimidade, principalmente através do ElGamal:




Nessa arquitetura, o Survey Designer é quem cria os questionários. Ele gera pares de chaves para o ElGamal e envia ao Survey Handler suas chaves públicas e também o questionário.

O Survey Handler por sua vez, irá repassar esses dados que ele recebeu para os nodos da rede, pedindo que eles respondam o questionário.

Os nodos, ao responderem o questionário, irão encriptar suas respostas com as chaves públicas do Survey Designer, enviando suas respostas para o Survey Handler, que irá esperar todos responderem e então aplicar as operações homomórficas de adição e multiplicação, obtendo o resultado final encriptado e então enviando de volta ao Survey Designer.

O Survey Designer irá então desencriptar os resultados com sua chave privada e não será possível estabelecer uma relação entre o que cada nodo teria respondido e o resultado final, ou seja, a anonimidade foi garantida nesse processo.

Note que o algoritmo do ElGamal varia um pouco se quisermos homomorfismo na adição ou na multiplicação, por isso o Survey Designer terá mais de uma chave pública. A adição pode ser usada para calcular respostas em que se queira a média, enquanto a multiplicação será utilizada para questões que possuírem alternativas (múltipla escolha).

Além disso, note que como um número aleatório é utilizado na encriptação, cifrar valores iguais (utilizando a mesma chave) gerará valores cifrados diferentes.

Também será possível que a resposta de uma pergunta seja uma *string*, nesse caso, o ElGamal não será utilizado, sendo necessário somente embaralhar as respostas.

Visando a simplicidade, supomos que todos os nodos são confiáveis e que se comunicam através de canais seguros e confiáveis.

## 5. RESULTADOS OBTIDOS

A aplicação foi implementada seguindo a descrição do tópico 4 e está disponível em https://github.com/Luan-Diniz/ElGamal-Application.

### 5.1 Backend

O backend da aplicação foi implementado em Python, utilizando a já citada LightPHE para o ElGamal, Flask [11] para a criação dos *endpoints* no qual as requisições de GET e POST são feitas (servindo como boa parte da comunicação desse sistema distribuído) e Pika [12] pois utilizamos o *message broker* RabbitMQ [13] para o Survey Handler distribuir o questionário para os Workers.

Cada nó representa um dos três papeis a seguir:

- Survey Designer
- Survey Handler
- Worker

Note que existe apenas um Survey Designer, um Survey Handler e um número arbitrário de Workers (na imagem do tópico 4 eles foram representados como Node 1, Node 2, … ,Node K).

O Survey Designer está no diretório ./services/survey-designer. Construímos ele de forma simples, mas de forma que ainda represente a arquitetura desejada. Desse modo, a lógica dele está toda no arquivo .py, com o qual é possível interagir durante a execução, já que ele pergunta quando deve enviar o questionário e quando deve pedir pelos resultados. O arquivo survey.json armazena a pesquisa que será utilizada.

As funcionalidades deste nodo são as de criar os pares de chaves do ElGamal, enviando o questionário e as chaves públicas para o Survey Handler. Além disso, utilizará suas chaves privadas para desencriptar os resultados finais recebidos, os quais serão processados e impressos para o usuário ver.

Sobre o survey.json, ele segue seguinte padrão:

```
{
  "Pergunta1" : [1, "average"],
  "Pergunta2" : [2, "multiple choice" ,
        {
        "a" : ["textoAlternativa1", 3],
        "b" : ["textoAlternativa2", 5],
        "c" : ["textoAlternativa3", 7],
        } ],
  "Pergunta3" : [3, "text"]
}
```

É um json no qual a chave é a pergunta, e o conteúdo é uma lista, sendo o primeiro número o *id* da pergunta e o segundo o tipo dela. As perguntas podem ser

para obter-se a média (“average”), uma resposta em texto (“texto”) ou o Worker escolher uma alternativa em uma questão de múltipla escolha (“multiple choice”). Neste último caso, é associado um número primo único na questão para cada alternativa. Esse número será usado quando a alternativa referente a ele for escolhida. Todas as respostas de uma questão de múltipla escolha são multiplicadas, o que gera um número que é fatorado de forma única em números primos. Essa fatoração ocorre no Survey Designer, e, tendo os números primos e suas potências obtidas na fatoração, então temos também a quantidade de vezes que cada alternativa naquela questão foi escolhida.

O Survey Handler se encontra no diretório ./services/survey-handler. Ele recebe as informações do Survey Designer e as repassa para todos os Workers através do message broker. Além disso, ele é o responsável por receber a resposta de cada Worker, armazenando-as, após realizar as operações homomórficas necessárias e enviando-as, quando solicitado, ao Survey Designer.

O Worker se encontra em ./services/workers, na pasta react se encontra o frontend, enquanto no /src está o backend. O backend recebe as informações do Survey Handler e as transmite ao frontend, onde o usuário irá responder a pesquisa.

### 5.2 Frontend

O frontend foi implementado utilizando React [14], uma biblioteca frontend da linguagem JavaScript, juntamente com o servidor de desenvolvimento local Vite.

Para estruturar a interface, é utilizado um questionário em formato JSON enviado por um worker. Após o usuário clicar no botão escrito “Submit”, será enviado um arquivo em JSON com as respostas obtidas para o worker que enviou o questionário.

## 6. EXECUTANDO A APLICAÇÃO

Primeiramente, é preciso preparar o *virtual enviroment* por meio da execução dos seguintes comandos na raiz dos diretórios da aplicação:

***1. python -m venv venv***
***2. source venv/bin/activate***
***3. pip install -r requirements.txt***
***4. deactivate  # Para sair do virtual enviroment***

Com o ambiente configurado, o próximo passo consiste em executar o ***rabbitmq***:

***1. docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management***

Agora que o ***rabbitmq*** está rodando, é preciso executar o survey-designer. Para isso, basta:

***1. python3 services/survey-designer/survey_designer.py***

O survey-designer irá enviar o questionário para o survey-handler. Então para o próximo passo:

***1. python3 services/survey-handler/survey_handler.py***

Nesse instante, será preciso rodar a quantidade desejada de workers em diferentes portas:

***1. python3 services/workers/src/worker.py WORKER_PORT***

Por fim, para cada worker, é necessário rodar o frontend:

***1. VITE_API_PORT=WORKER_PORT npm dev run FE_PORT***

O valor da flag ***VITE_API_PORT*** deve ser a porta do respectivo worker enquanto ***FE_PORT*** é a porta em que o frontend irá executar.

## 7. DEMONSTRAÇÃO DA APLICAÇÃO

O questionário que desejamos enviar é estruturado em formato JSON da seguinte forma:

```
{
    "Qual seu salário?" : [1, "average"],
    "O quão satisfeito você está com o ambiente na empresa?" : [2, "multiple choice" ,
        {
            "a" : ["Muito satisfeito", 3],
            "b" : ["Satisfeito", 5],
            "c" : ["Nem insatisfeito, nem satisfeito", 7],
            "d" : ["Insatisfeito", 11],
            "e" : ["Muito insatisfeito", 13]
        }
    ],

    "Você precisa fazer horas extra frequentemente?" : [3, "multiple choice" ,
        {
            "a" : ["Sim", 3],
            "b" : ["Não", 5]
        }
    ],

    "O quão bem sua saúde mental está?" : [4, "multiple choice" ,
        {
            "a" : ["Muito bem", 3],
            "b" : ["Bem", 5],
            "c" : ["Poderia estar melhor", 7],
            "d" : ["Mal", 11],
            "e" : ["Muito mal", 13]
        }
    ],

    "Alguma observação?" : [5, "text"]
}
```

Ao executar o survey designer, ele aguarda a confirmação do envio do questionário

```
(venv) pedro@pedro-Dell-G15-5515:~/ElGamal-Application/services/survey-designer$ python3 survey_designer.py
WARNING:
Make sure you are executing this .py either from the root directory either from the /survey-designer directory!
Creating keys...
Do you want to send the survey? y/n
```

E ao executar o worker, ele aguarda para recebê-lo

```
(venv) pedro@pedro-Dell-G15-5515:~/ElGamal-Application/services/workers/src$ python3 worker.py
You can pass a arg to worker.py that will configure the PORT that this backend will run on.
WARNING:
Make sure you are executing this .py either from the root directory either from the /workers directory!
Waiting for the survey...
```

Ao acessar localhost:5173/answer_form, temos a interface gráfica do questionário indicando o aguardo




Logo, é enviado o questionário desejado

```
(venv) pedro@pedro-Dell-G15-5515:~/ElGamal-Application/services/survey-designer$ python3 survey_designer.py
WARNING:
Make sure you are executing this .py either from the root directory either from the /survey-designer directory!
Creating keys...
Do you want to send the survey? y/n
y
Sending survey...
Response data:  {'status': 'working'}
You want the results? y/n
```

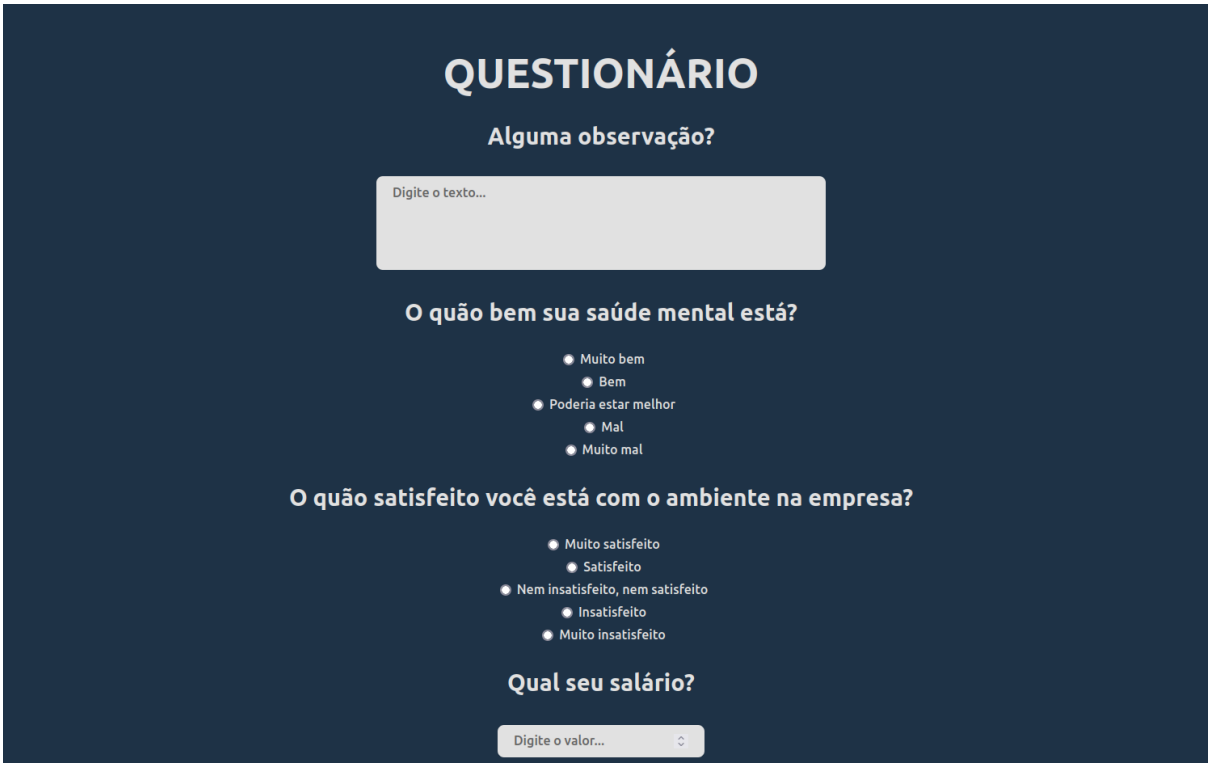
Captura do questionário pelo survey handler

```
^C(venv) pedro@pedro-Dell-G15-5515:~/ElGamal-Application/services/survey-handler$ python3 survey_handler.py
 * Serving Flask app 'survey_handler'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:4997
Press CTRL+C to quit
Received a survey!
127.0.0.1 - - [01/Jul/2024 21:52:00] "POST /survey HTTP/1.1" 200 -
```

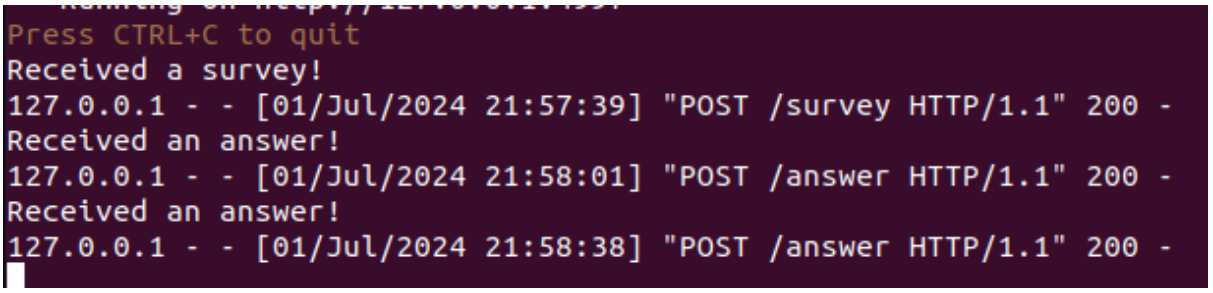
Recebimento por parte do worker

```
(venv) pedro@pedro-Dell-G15-5515:~/ElGamal-Application/services/workers$ python3 src/worker.py
You can pass a arg to worker.py that will configure the PORT that this backend will run on.
WARNING:
Make sure you are executing this .py either from the root directory either from the /workers directory!
Waiting for the survey...
Received a survey to answer!
 * Serving Flask app 'worker'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

A interface agora é atualizada, mostrando as perguntas do questionário




Ao finalizar o preenchimento, o questionário pode ser enviado. Aqui temos o envio de duas respostas para o survey handler.




Quando o survey designer pedir pelos resultados, ele apresenta os valores para quem enviou o questionário, após serem processados pelo survey handler. Assim, não é possível saber que usuários deram quais respostas, somente o resultado das operações realizadas sobre os dados criptografados pelo handler.

```
----------SURVEY RESULTS----------
Qual seu salário?
Average answer: 2750.0

O quão satisfeito você está com o ambiente na empresa?
Alternatives:
a: Muito satisfeito
        The alternative above was chosen 0 times.
b: Satisfeito
        The alternative above was chosen 1 times.
c: Nem insatisfeito, nem satisfeito
        The alternative above was chosen 1 times.
d: Insatisfeito
        The alternative above was chosen 0 times.
e: Muito insatisfeito
        The alternative above was chosen 0 times.

Você precisa fazer horas extra frequentemente?
Alternatives:
a: Sim
        The alternative above was chosen 2 times.
b: Não
        The alternative above was chosen 0 times.

O quão bem sua saúde mental está?
Alternatives:
a: Muito bem
        The alternative above was chosen 0 times.
b: Bem
        The alternative above was chosen 0 times.
c: Poderia estar melhor
        The alternative above was chosen 1 times.
d: Mal
        The alternative above was chosen 1 times.
e: Muito mal
        The alternative above was chosen 0 times.

Alguma observação?
Answers:
        Estou legal
        Estou com muita dor

----------END OF SURVEY----------
```

## 8. REFERÊNCIAS BIBLIOGRÁFICAS


[1] WIKIPEDIA, **ElGamal encryption**. Wikipedia, 2024. Disponível em: https://en.wikipedia.org/wiki/ElGamal_encryption. Acesso em: 15 jun. 2024.

[2] ELGAMAL, T. **A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms.** Lecture Notes in Computer Science, pp. 10-18, 1985.

[3] SERENGIL, SEFIK ILKIN. **A Step by Step Partially Homomorphic Encryption Example with ElGamal in Python.** Sefiks, 2023. Disponível em: https://sefiks.com/2023/03/27/a-step-by-step-partially-homomorphic-encryption-example-with-elgamal-in-python/. Acesso em: 15 jun. 2024.

[4] SERENGIL, SEFIK ILKIN. **LightPHE Library**. GitHub, 2024. Disponível em: https://github.com/serengil/LightPHE. Acesso em: 15 jun. 2024.

[5] CRYPTO STACK EXCHANGE. **Can Elgamal be made additively homomorphic and how could it be used for E-voting?** Stack Exchange, 2019. Disponível em: https://crypto.stackexchange.com/questions/3626/can-elgamal-be-made-additively-homomorphic-and-how-could-it-be-used-for-e-voting. Acesso em: 15 jun. 2024.

[6] NVOTES BLOG. **Multiplicative vs Additive Homomorphic ElGamal**. nVotes, 2020. Disponível em: https://nvotes.com/multiplicative-vs-additive-homomorphic-elgamal/. Acesso em: 15 jun. 2024.

[7] IEEE DIGITAL PRIVACY. **Homomorphic Encryption Use Cases.** IEEE, 2024. Disponível em: https://digitalprivacy.ieee.org/publications/topics/homomorphic-encryption-use-cases#:~:text=Through%20homomorphic%20encryption%2C%20cloud%20providers,where%20sensitive%20data%20is%20shared.

[8] STINSON and PATERSON. **Cryptography: Theory and Practice**, fourth edition, 2018.

[9] PINTER, CHARLES C.. **A Book Of Abstract Algebra**, second edition, Dover Books On Mathematics, 2019.

[10] ROCHA, V. F. and LÓPEZ, JULIO. **An overview on homomorphic encryption algorithms.** Technical Report, UNICAMP, 2018.

[11] PALLETS PROJECTS. **Flask Documentation.** Disponível em: https://flask.palletsprojects.com/en/3.0.x/. Acesso em: 29 jun. 2024.
[12] PIKA TEAM. **Pika Documentation.** Disponível em: https://pika.readthedocs.io/en/stable/. Acesso em: 29 jun. 2024.
[13] RABBITMQ. **RabbitMQ Documentation.** Disponível em: https://www.rabbitmq.com/. Acesso em: 29 jun. 2024.
[14] REACT. **React Documentation.** Disponível em: https://react.dev/. Acesso em: 29 jun. 2024.